

Middleware & Architecture

1. What are the advantages of using middleware in an Express application?
 - ⇒ Allow developers to handle the section between request and response.
2. How does separating middleware into dedicated files improve the maintainability of your code?
 - ⇒ By promoting modularity this allows developers to group the correlated functions as a component.
3. If you had to scale this API to support user roles (e.g., admin vs student), how would you modify the middleware structure?
 - ⇒ Modify it to be a reusable block like authentication.

Query Handling & Filtering

4. How would you handle cases where multiple query parameters conflict or are ambiguous (e.g., `minCredits=4` and `maxCredits=3`)?
 - ⇒ It would depend on the type of conflict. For the range like credit, I check whether the min value is greater than the max? if it is, return 400, indicate that an error has occurred.
5. What would be a good strategy to make the course filtering more user-friendly (e.g., handling typos in query parameters like “falll” or “dr. smtih”)?
 - ⇒ I would turn the query, and the data to lower or upper case first, then use the include method, not to compare the exact value.

Security & Validation

6. What are the limitations of using a query parameter for authentication (e.g., `?token=xyz123`)? What alternatives would be more secure?
 - ⇒ by using a query parameter, everyone can see it in the URL, and the text is not encrypted.

For the alternative approach is to use Authorization Header

Instead of putting the token in the URL, send it in the request header.

7. Why is it important to validate and sanitize query inputs before using them in your backend logic?

⇒ It is important to catch the error or bad data before using them in the backend logic, otherwise the system will crash or have unexpected behavior.

Abstraction & Reusability

8. Can any of the middleware you wrote be reused in other projects? If so, how would you package and document it?

⇒ Yes, files like login.js and auth.js are general and can be reused anywhere if it suits.
⇒ To reuse them, I would organize them into groups, and write clear comments explaining what each one does and how to use it.

9. How could you design your route and middleware system to support future filters (e.g., course format, time slot)?

⇒ I would use a dynamic approach instead of fixed filtering. This way would let the core filtering logic untouched whenever a new item is added to the list.

Bonus – Real-World Thinking

10. How would this API behave under high traffic? What improvements would you need to make for production readiness (e.g., rate limiting, caching)?

⇒ Under high traffic, current API would slow down since it filters an array on every request with no limit. For the production, I would add a limit, and move data to a real database.